Reproducing RPM-MCTS for Code Generation

Knowledge-Retrieval as Process Reward Model with Monte Carlo Tree Search

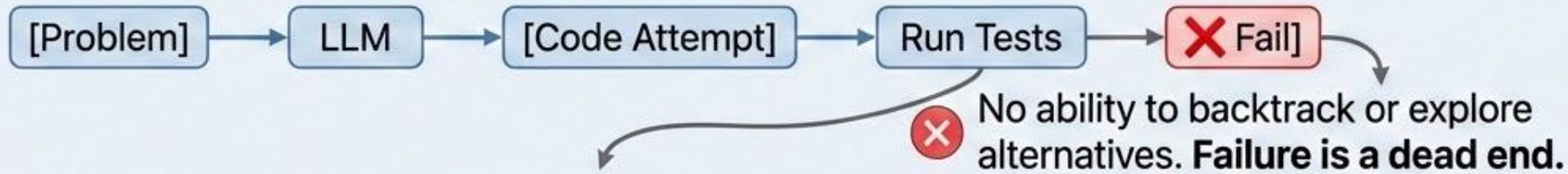
accepted by AAAI 2026.
https://arxiv.org/pdf/2511.19895

Xiaorui Liu

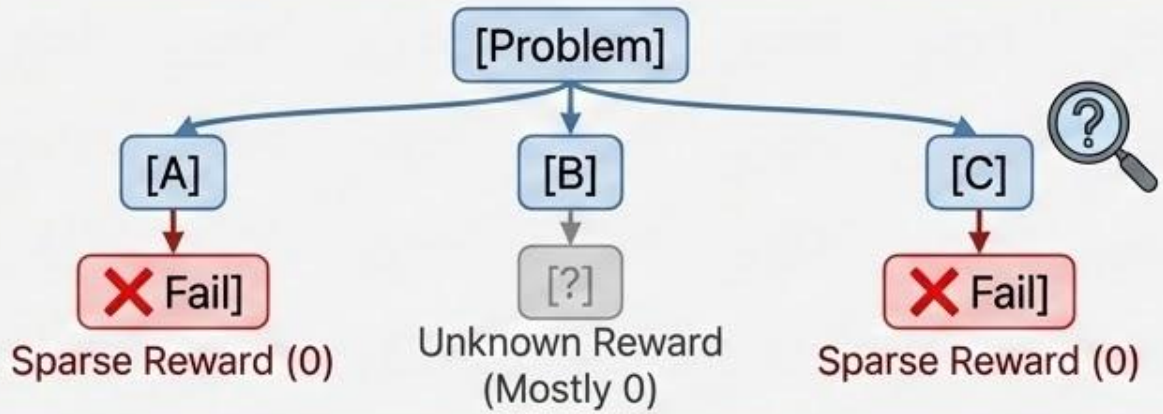
The Problem: Standard LLMs Generate Code in One Shot

From blind generation to guided search with RPM-MCTS

THE PROBLEM:
Linear, "One-Shot" Generation Fails

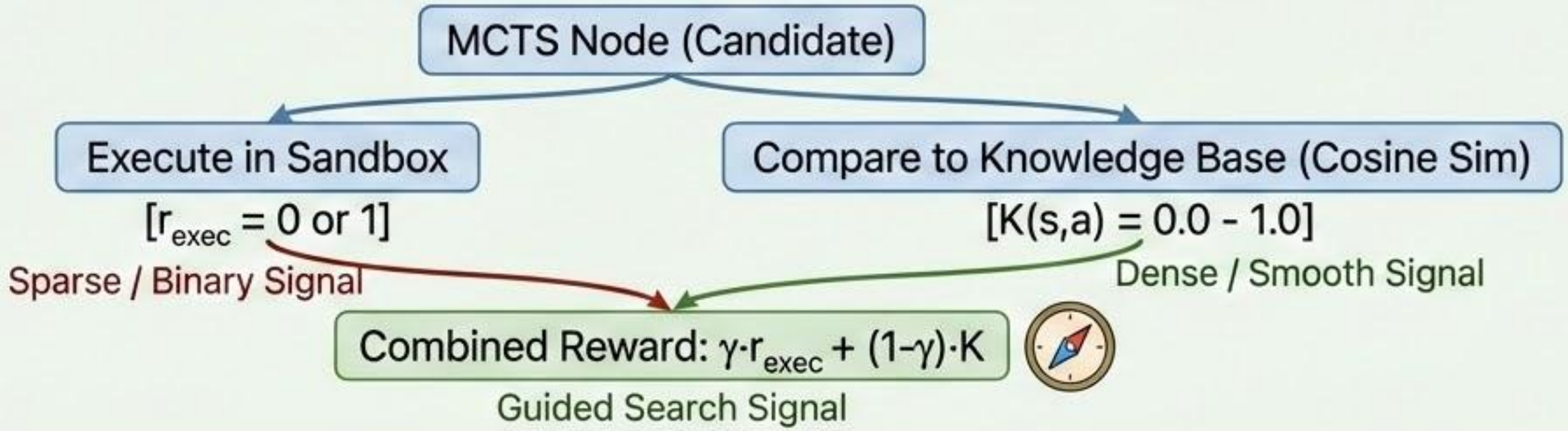


THE IDEA & CORE CHALLENGE:
MCTS Explores, But Search is "Blind"



Most candidates fail tests, providing almost no reward signal. The search tree becomes unguided and **blind.**

THE SOLUTION:
RPM-MCTS & Dense Knowledge Base Rewards



The Four Phases of RPM-MCTS

5 simulations × 3 branches = up to 15 candidates tested per problem.

1. SELECT

Input: Tree root
Action: Walk down tree picking child with highest $Q + \beta \cdot UCB + \alpha \cdot K$
Output: Leaf node to expand

2. EXPAND

Input: Leaf node
Action: LLM generates 3 candidates sequentially (each sees prior siblings). Prune duplicates with cosine sim > 0.85
Output: 1–3 new child nodes

3. EVALUATE

Input: New child
Action: Execute code in sandbox. If fail → truncate to good prefix → inject verified prefix as new node
Output: exec_score (0 or 1) + feedback

4. BACKUP

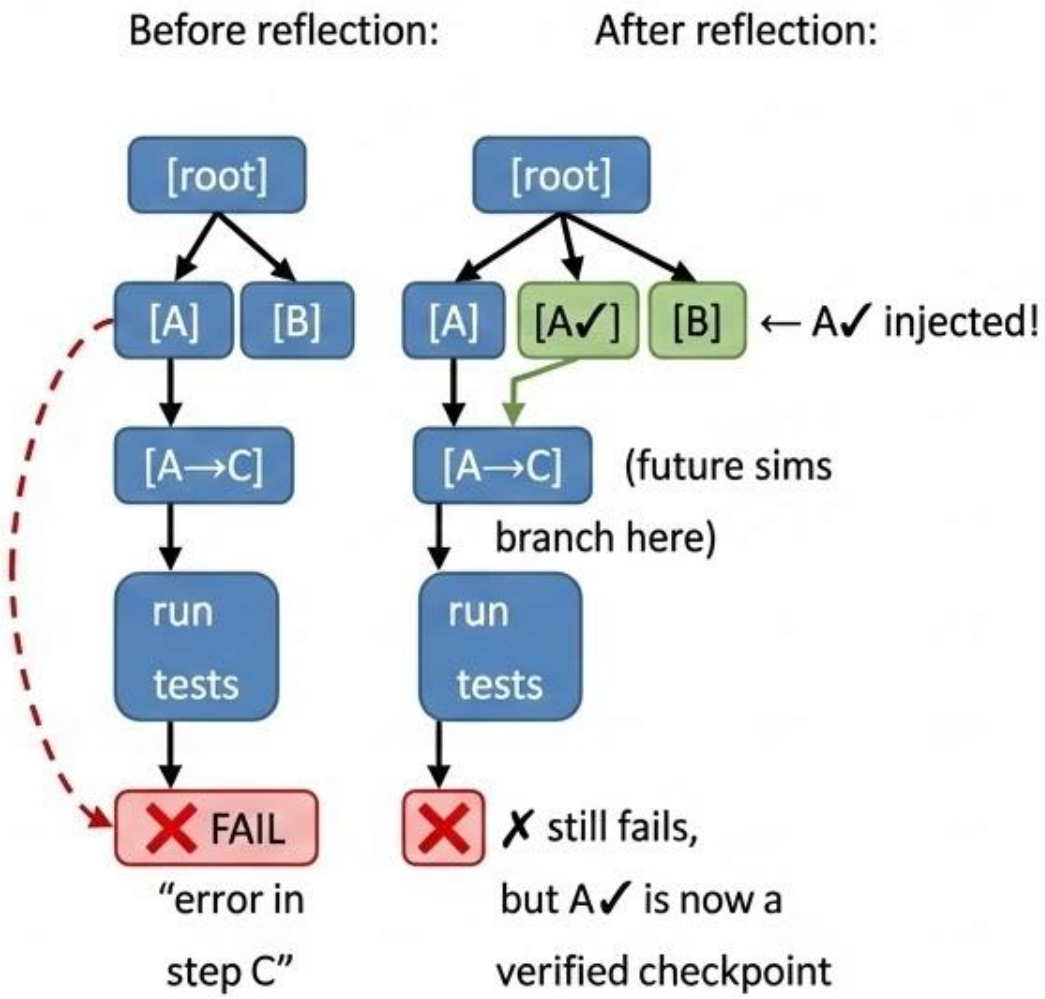
Input: Score
Action: reward = $\gamma \cdot r_{exec} + (1-\gamma) \cdot K(s,a)$ → update $Q(s,a)$ and $N(s,a)$ for all ancestors
Output: Updated tree values

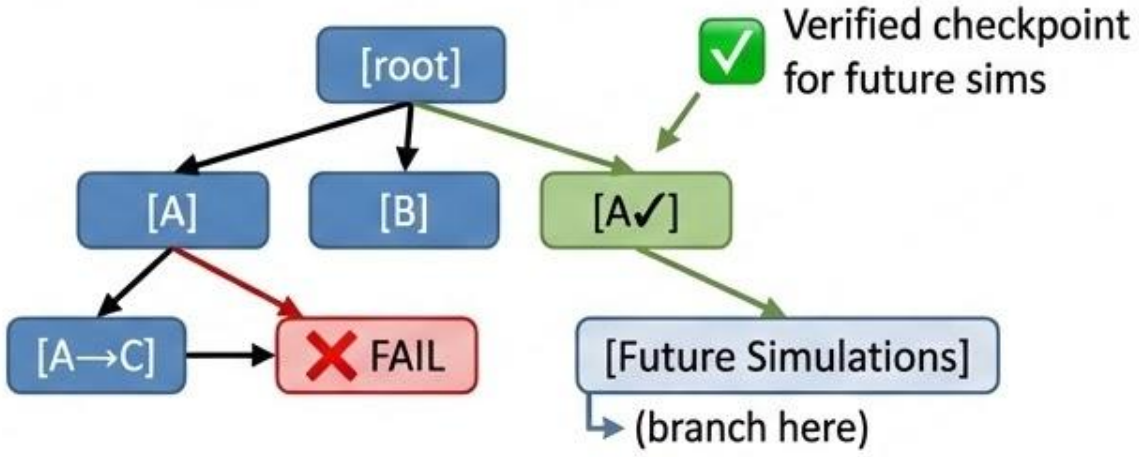
Repeat for each simulation

Paper hyperparameters:

β (exploration)	α (KB weight)	γ (exec vs KB)	Sim threshold	Simulations	Branches
0.5	0.5	0.5	0.85	5	3

Reflection & Dynamic Node Injection





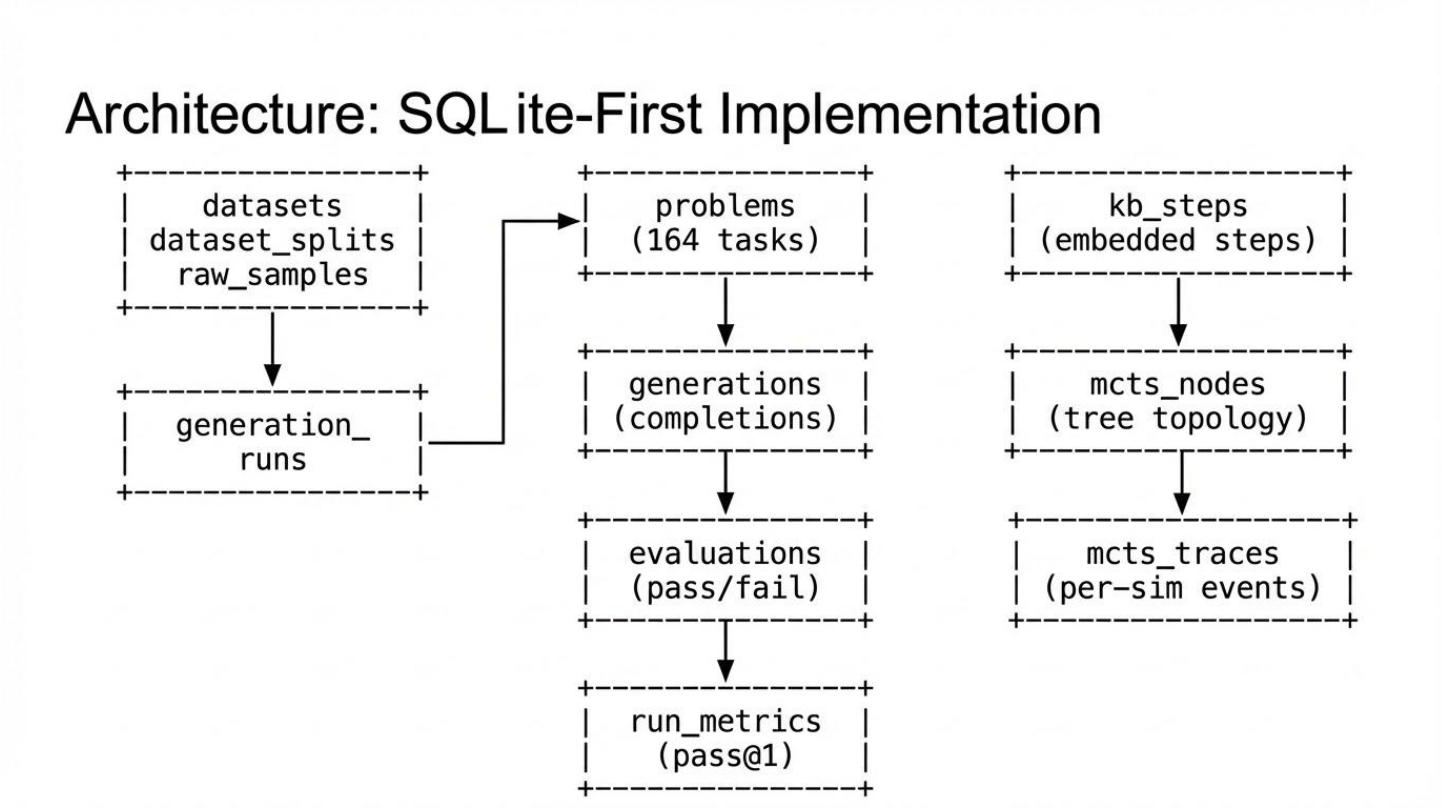
How it works:

1. Code fails → identify the erroneous step (last step)
2. Truncate to the "good prefix" (everything before the error)
3. Verify the prefix executes without error
4. **Inject prefix as a new node** in the tree → future simulations branch from it

Implementation Pipeline

Pipeline:

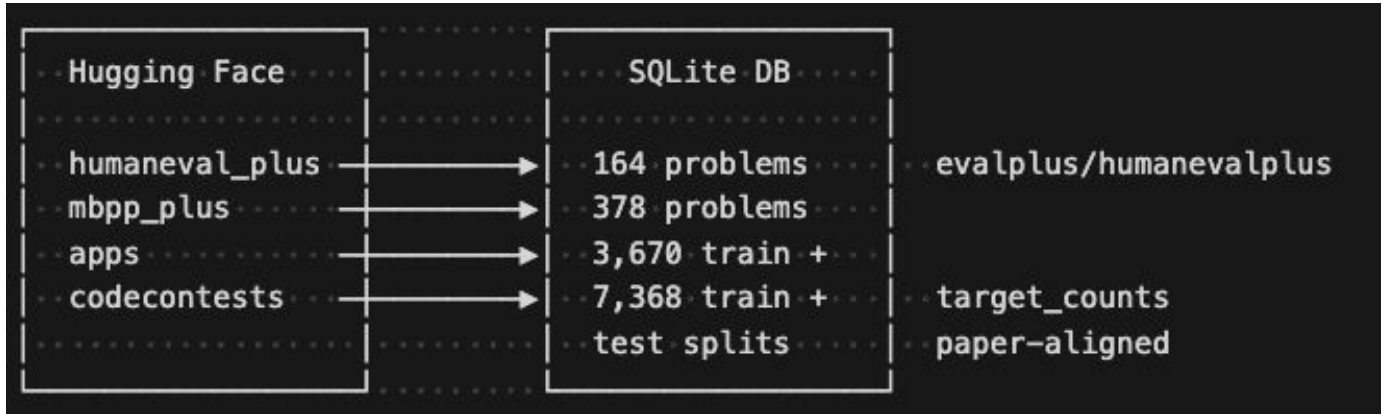
```
ingest ──→ build-kb ──→ generate-baseline ──→ evaluate ──→ report
   │                          ▲
   └──────→ run-rpm-mcts ──→ evaluate ──┘
```



1. **SQLite-first:** All artifacts in DB — no intermediate files, and makes every artifact auditable and reproducible
2. **CLI entrypoints**: commands only, fixed DB path — reproduce from scratch
3. **run_name**: Unique per run— SQL traceable
4. **DB schema — 4 core tables**
   - **problems:** prompt, entry_point, test_spec_json (HF/EvalPlus → SQLite) |
   - **generations**: run_id, problem_id, prompt_text, completion_text (per run_name)
   - **evaluations**: generation_id, passed (0/1), stdout, stderr (harness output)
   - **mcts_traces**: run_id, problem_id, payload_json (exec_score per simulation) |

Reproduce: Ingestion

- Ingestion is deterministic: fixed source, explicit filtering, and fixed target_counts per split to match the paper's benchmark sizes.
- Ingestion writes raw records into raw_samples and runnable tasks into problems in one transaction.




```python
# dataset_catalog.py — raw_keep_fields + target_counts
DatasetSpec(hf_path="deepmind/code_contests",
    raw_keep_fields=["name", "description", "solutions"],  # drop 4GB test I/O
    target_counts={"train": 7368, "test": 150})
```

Reproduce: Baseline Generation + Evaluation Harness

For each of 164 problems:

[Function stub + docstring] ——→ gpt-4o-mini ——→ [Completion] ——→ Run tests

Evaluation harness — PASS_MARKER

```
# evaluate.py — tests inject marker; pass = returncode 0 AND marker in stdout
PASS_MARKER = "__RPM_BASELINE_PASS__"
passed = int(proc.returncode == 0 and PASS_MARKER in proc.stdout)
```

| Calls per problem | 1 |

| Result| pass@1 = 33.54% (55/164)|

Reproduce: Knowledge Base Construction

```
Training data (APPS + CodeContests)
        |
        ▼
┌──────────────────────┐
│ For each solution:   │
│ Split into logical   │ ──→ "sort the input array"
│ steps (~8 lines ea)  │ ──→ "use two pointers from both ends"
│                      │ ──→ "check boundary conditions"
└──────────────────────┘
        |
        ▼
┌──────────────────────┐
│ Embed each step      │     sentence-transformers/
│ with MiniLM-L6-v2    │ ──→ all-MiniLM-L6-v2
│ → 384-dim vector     │     (384 dimensions)
└──────────────────────┘
        |
        ▼
┌──────────────────────┐
│ Store in kb_steps    │     step_text + embedding_blob
│ table in SQLite      │     → ready for retrieval
└──────────────────────┘
```

At search time: `K(s,a) = max cosine_sim(node, all_kb_steps)`

```python
# kb.py — split solution into ~8-line steps, embed each
def _solution_to_steps(solution, max_steps=8, max_chars=400):
    # break on structural keywords: def, for, while, if, class
    # → yields list of code step strings

# kb.py — retrieve KB reward for an MCTS node
class KBRetriever:
    def __init__(self, conn, embedding_model):
        # load all kb_steps embeddings from SQLite into a numpy matrix
        self.matrix = np.stack(vectors)          # shape: (num_steps, 384)

    def max_similarity(self, text, top_k=5):
        q = self.embedder.encode([text])[0]      # embed the MCTS node text
        sims = self.matrix @ q                   # cosine sim with all KB steps
        return float(max(sims)), top_texts       # K(s,a) = max similarity
```

Reproduce: MCTS

```python
@dataclass
class Node:
    text: str                    # state s — full candidate code or cumulative steps
    depth: int
    kb_reward: float             # K(s,a) — cosine sim to KB
    parent: "Node | None" = None
    visits: int = 0              # N(s,a)
    value_sum: float = 0.0       # for computing Q(s,a) = value_sum / visits
    children: list["Node"] = field(default_factory=list)
    action_text: str = ""        # single action a this node adds over parent
    is_pruned: bool = False      # filtered by similarity pruning?

    def q(self) -> float:        # Q(s,a) = empirical mean reward
        return self.value_sum / self.visits if self.visits > 0 else 0.0
```

Reproduce: MCTS

Phase 1: Selection (UCB + α·K)

```python
def _ucb(parent, child, cfg):
    # Paper Eq (3)+(4): SelectionScore = UCB + α · K(s,a)
    explore = cfg.ucb_beta * math.sqrt(math.log(parent.visits + 1) / (child.visits + 1e-6))
    return child.q() + explore + cfg.alpha_select * child.kb_reward
    #      ↑ exploit     ↑ explore (UCB)   ↑ KB process reward

def _select_leaf(root, cfg):
    # walk tree picking highest-scoring child at each level
    node = max(candidates, key=lambda c: _ucb(parent, c, cfg))
```

Term	Meaning	Paper symbol
`child.q()`	Empirical mean reward (exploitation)	Q(s,a)
`explore`	UCB exploration bonus	β·√(log N(s) / N(s,a))
`child.kb_reward`	Cosine similarity to KB	K(s,a)

Reproduce: MCTS

```
# Phase 2: Expansion (diversity + sim filter)

```python
def _expand(node, prompt, entry_point, backend, retriever, cfg, ...):
    for b in range(cfg.branching_factor):      # diversity: each sibling sees prior
        # instruction BEFORE stub (avoids echo bug — see Issue 1)
        instruction = f"Complete the following Python function. "
                      f"Return ONLY the function body for def {entry_point}.\n\n"
        base = instruction + sibling_ctx + prompt
        result = backend.generate(prompt=base, ...)
        generated_siblings.append(result["completion"])  # next sibling sees this

    # similarity filter: prune near-duplicates (cosine > 0.85)
    kept, pruned = _similarity_filter(raw_candidates, retriever, cfg.similarity_threshold)

    # create child nodes — KB score for selection only, NOT Q-value
    for action, full_state in kept:
        kb_score, _ = retriever.max_similarity(action, top_k=3)
        child = Node(text=full_state, kb_reward=kb_score, value_sum=0.0)
        _persist_node(conn, child, run_id, problem_id)
```
```

Reproduce: MCTS

```
# Phase 3: Evaluation + Lock Winner

```python
def _reflect_and_score(dataset_name, prompt, entry_point, node, ...):
    # sanitize raw LLM output → executable code (5-stage pipeline)
    code_text, _ = sanitize_humaneval_candidate(prompt, node.text, entry_point)
    # run code + tests in a subprocess
    res = execute_problem(dataset_name, prompt, code_text, entry_point, ...)

    if int(res["passed"]) == 1:   return code_text, 1.0, "", []    # pass
    if "SyntaxError" in feedback: return code_text, 0.0, feedback, []
    return code_text, 0.2, feedback, []                            # logic error

def _run_python(program_text, timeout_sec):
    # write to temp file → subprocess.run → check returncode + PASS_MARKER
    passed = int(proc.returncode == 0 and PASS_MARKER in proc.stdout)
```
```

Reproduce: MCTS

```
# Phase 4: Backprop + Main Loop

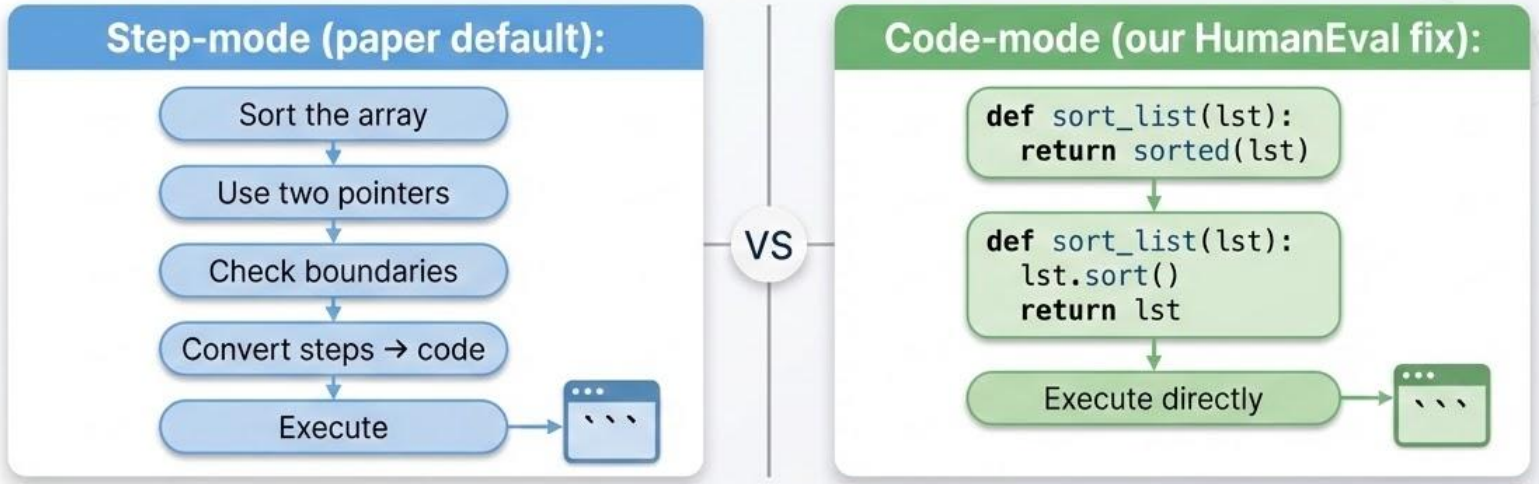
```python
def _backpropagate(path, reward, conn):
    for p in path:              # update every node root → leaf
        p.visits += 1;  p.value_sum += reward

def _search_one(prompt, entry_point, backend, retriever, cfg, ...):
    best_code, best_exec = "", -1.0

    for sim_idx in range(cfg.num_simulations):          # 5 simulations
        node, path = _select_leaf(root, cfg)            # ① Selection
        children = _expand(node, prompt, ...)           # ② Expansion
        code_text, exec_score, ... = _reflect_and_score(node, ...)  # ③ Evaluation

        reward = cfg.gamma_backup * exec_score \        # ④ Backprop
                 + (1 - cfg.gamma_backup) * node.kb_reward
        _backpropagate(path, reward, conn)

        if exec_score > best_exec:                      # lock winner by exec score
            best_exec, best_code = exec_score, code_text
        if exec_score == 1.0:                           # early exit → no dropped
            return best_code, best_exec, traces
```
```

Issues Encountered & How We Solved Them

Problem: Raw LLM output ≠ executable code.

5-Stage Sanitization Pipeline:

Stage	Function	What it does
1	` _extract_code_from_markdown_fences`	Strip ```python ``` wrappers
2	` _remove_forbidden_solution_lines`	Drop `assert`, `main()`, `check_solution()`
3	` _extract_entrypoint_function_block`	AST-based: keep only `def entry_point(...)`
4	` _reindent_completion`	Restore indentation lost by `.strip()`
5	`_sanitize_humaneval_candidate`	Assemble full function if needed

Raw gpt-4o-mini output:

```
"Here's the implementation:
```python
def has_close_elements(...):
    for i, n1 in enumerate(...):
        ...
```

You can test it with..."
```

After sanitization pipeline:

```python
def has_close_elements(numbers, threshold):
    for i, n1 in enumerate(numbers):
        for j, n2 in enumerate(numbers):
            if i != j and abs(n1-n2) < threshold:
                return True
    return False
```

Issues Encountered & How We Solved Them

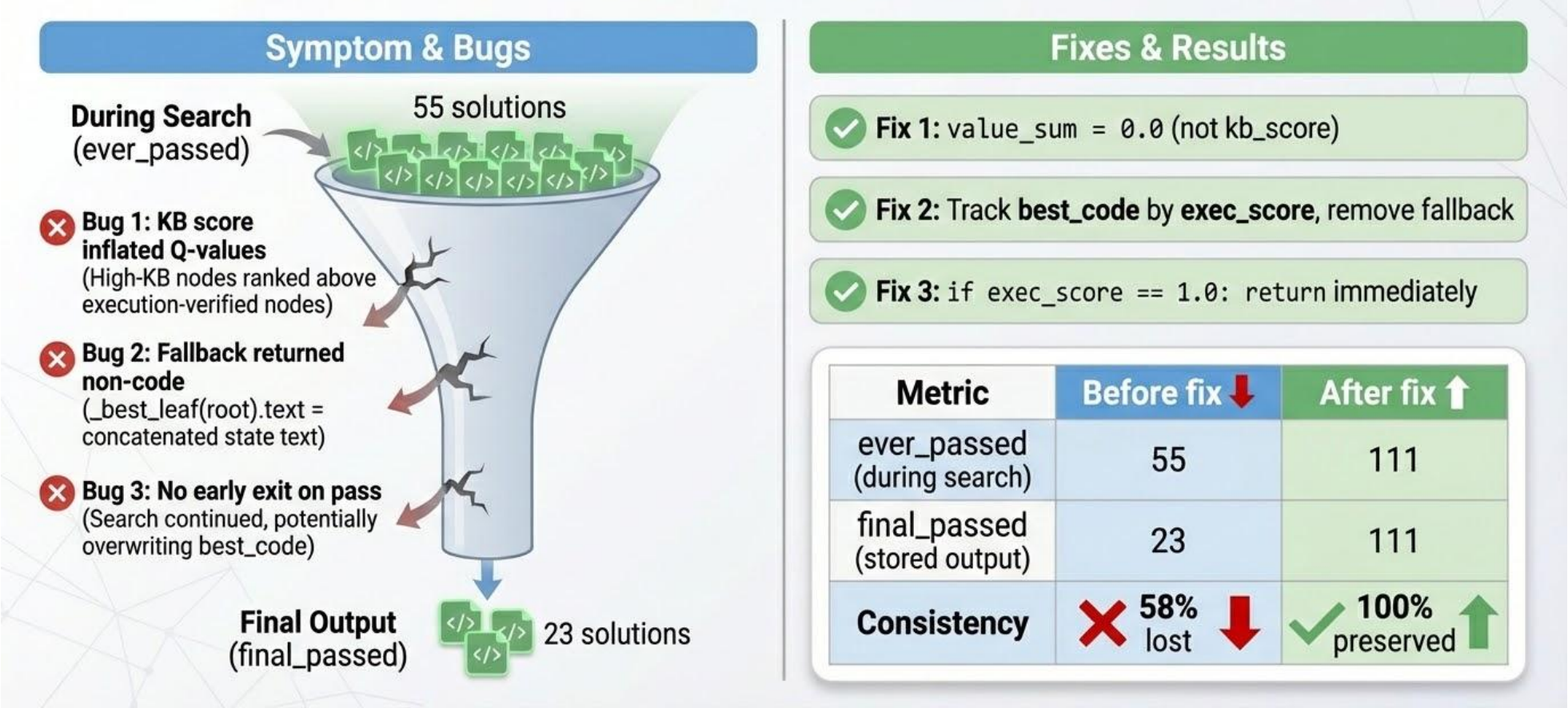


Feature	Step-mode	Code-mode
Datasets	APPS, CodeContests	HumanEval+
Node content	Natural-language step	Full Python function
Expansion prompt	Next algorithmic step	Complete this function
Reflection	Truncate + inject prefix	Disabled (`reflection_iters=0`)
Node injection	Yes (verified prefixes)	No (no step prefixes)

Issues Encountered & How We Solved Them

Problem: ever_passed ≠ final_passed

Symptom: 55 solutions passed during search → only 23 in final output.



Performance: HumanEval+

	Baseline	RPM-MCTS	Δ	Relative
gpt-4o-mini (ours)	**33.54%** (55/164)	**67.68%** (111/164)	**+34.14pp** ↑	**2.02x** ↑
Qwen3-8B (paper)	75.6%	86.0%	+10.4pp ↑	1.14x ↑
Qwen3-235B (paper)	88.4%	93.3%	+4.9pp ↑	1.06x ↑
Claude 3.7 (paper)	96.3%	98.2%	+1.9pp ↑	1.02x ↑

RPM-MCTS achieves a remarkable 2.02x performance boost for **gpt-4o-mini**, significantly outperforming the relative gains of larger models.